

프로젝트 완료 보고서

- MicroBlaze AXI4-Lite SoC 설계 -

작성자 정광근

작성일 2026.06.30

진행 기간 2026.06.22 ~ 2026.06.30

목차

1. 개요 (Overview).....	4
1.1. 목적 및 목표	4
1.2. 설계 사양 (Specification).....	4
2. 프로젝트 관리 (Project Management)	5
2.1. 일정 계획 (Schedule)	5
2.2. 설계 환경 (Design Environment)	5
3. 아키텍처 설계 (Architecture).....	6
3.1. 시스템 구조.....	6
3.2. 시스템 기능 (Function).....	6
3.3. 소프트웨어 계층 구조 (Vitis Layer Structure).....	6
3.4. 설계 이론 및 배경 (Theory & Background)	6
4. 상세 설계 (Detailed Design).....	13
4.1. Custom GPIO IP.....	13
4.2. Custom Timer IP.....	13
4.3. Custom UART IP.....	13
4.4. Custom I2C Master IP.....	13
4.5. Custom SPI Master IP	13
5. UVM 검증 환경 설계 (GPIO IP Verification).....	23
5.1. 검증 대상 및 목표.....	23
5.2. Testbench 구성 요소.....	23
5.3. Driver 설계	23
5.4. Scoreboard 설계	23
5.5. Coverage 설계	23
5.6. Sequence 시나리오	23
5.7. Testbench Top.....	23
6. 시뮬레이션 및 검증 (Simulation & Verification).....	27
6.1. GPIO IP UVM 검증 결과.....	27
6.2. 보드 통합 검증 (Bring-up & Verification).....	27

7. 결과 분석 및 트러블슈팅 (Analysis & Troubleshooting).....	29
7.1. Implementation.....	29
7.2. Troubleshooting 1 — AXI 템플릿 Read Data(rdata) 출력 누락.....	29
7.3. Troubleshooting 2 — RC522 SPI Address Byte 포맷 규칙 혼선.....	29
7.4. Troubleshooting 3 — 모드 전환 시 LCD 화면 글자 겹침.....	29
7.5. Troubleshooting 4 — SR04 측정값 불안정(타이머 기반 로직 전환).....	29
7.6. Troubleshooting 5 — 타이머 고장(FND 8888 먹통).....	29
7.7. Troubleshooting 6 — REQA 직후 ProtocolErr 발생.....	29
7.8. Troubleshooting 7 — 비트스트림/XSA 플랫폼 불일치.....	29
7.9. Troubleshooting 8 — 부팅 순서 오류에 따른 시스템 락업.....	29
7.10. Troubleshooting 9 — RFID SPI FAIL (VER=0x00).....	29
7.11. Troubleshooting 10 — SR04 역배선 및 LCD 미점등.....	29
8. 결론 및 고찰 (Result & Discussion)	32
8.1. 결론	32
8.2. 고찰 및 개선 방안.....	32

1. 개요 (Overview)

1.1. 목적 및 목표

이번 프로젝트의 목적은 Basys3 FPGA 보드 위에 MicroBlaze와 AXI4-Lite 버스를 중심으로 SoC(System on Chip)를 구성하고, 직접 설계한 custom IP(GPIO, Timer, UART, I2C, SPI)를 통해 다양한 주변장치를 하나의 모드형 application으로 통합 제어하는 것이다. 기존 프로젝트에서는 각 기능이 개별 RTL 모듈과 FSM으로 구현되었지만, 이번에는 processor 기반 SoC 구조를 채택하여 하드웨어 IP와 소프트웨어 application의 역할을 분리하고, memory-mapped register를 통해 소프트웨어가 하드웨어를 제어하는 임베디드 시스템의 표준적인 개발 흐름을 경험하는 것을 목표로 하였다.

기능 측면에서는 SW[2:0] 스위치 조합에 따라 Stopwatch, 현재 시간 표시, RFID 근태관리, SR04 거리 측정, 시간 설정의 모드로 동작하는 임베디드 데모 콘솔을 구현하였다. 각 모드의 결과는 LCD1602(I2C), FND, LED, USB serial terminal을 통해 출력되며, 사용자는 보드 버튼과 terminal 명령을 통해 시스템을 제어할 수 있다.

또한 검증 측면에서는 핵심 블록 중 하나인 custom GPIO IP에 대해 UVM 기반 검증 환경을 구축하였다. sequence-driver-monitor-scoreboard-coverage 구조를 갖춘 테스트벤치를 통해 레지스터 접근, byte-strobe(wstrb) 처리, 방향 제어에 따른 io_port 동작을 자동 비교와 functional coverage로 체계적으로 검증하여, 보드 브링업 이전에 IP 수준의 신뢰성을 확보하는 것을 목표로 하였다.

1.2. 설계 사양 (Specification)

구분	설계 사양
System clock	100MHz
CPU	MicroBlaze soft-processor (local memory 128KB, standalone)
Bus	AXI4-Lite (AXI Interconnect 기반 memory-mapped I/O)
USB Serial	Xilinx axi_uartlite, 115200 bps (Vitis terminal)
Custom UART	custom AXI UART IP, PMOD JB1/JB2 (stopwatch r/c 통신)
Custom IP	GPIO x5 IPs, Timer, UART, I2C Master, SPI Master
System tick	custom Timer, PSC=99 / ARR=999 → 1ms interrupt
센서/주변장치	SR04(초음파, GPIO 제어), LCD1602(I2C), RFID-RC522(SPI)
동작 모드	SW[2:0] 기반 모드
Verification	UVM 기반 GPIO IP 검증 환경 (agent / scoreboard / functional coverage)

2. 프로젝트 관리 (Project Management)

2.1. 일정 계획 (Schedule)

기간	주요 작업
26.06.22	프로젝트 범위 정의, 기존 LCD/RFID/SPI 프로젝트 분석, AXI4-Lite 주소맵·핀맵 정리
26.06.23	Vivado block design 병합, custom IP(GPIO/Timer/UART/I2C/SPI) 연결, GPIOE 신설 및 XDC 반영
26.06.24	HAL 계층 설계(struct와 RTL 레지스터 맵 1:1 매핑), ModeManager 및 LCD ownership 설계, Startup 순서 정리
26.06.25	SPI/RC522 캡슐화(SS_HOLD 은닉) 및 VersionReg 진단 구현, SR04 타이머 기반 측정 로직·5회 중간값 필터 적용
26.06.26	RFID 카드 인식 로직(Transceive/REQA/Anticollision, 1.5초 쿨다운) 완성, GPIO IP UVM 검증 환경 구축
26.06.27	UVM Scoreboard pass/fail 판정 로직 완성, Regression 실행(Fail 0건, Coverage 100%),
26.06.28	빌드 검증, 보드 bring-up 및 트러블슈팅
26.06.29	최종 통합 검증(모드 전환·카드 분기·쿨다운), 완료보고서 초안 및 레지스터맵 Reference Manual(34개 레지스터) 작성
26.06.30	발표 및 최종 산출물 정리

2.2. 설계 환경 (Design Environment)

분류	내용
개발 언어	Verilog / SystemVerilog(UVM), C (Vitis BareMetal application)
FPGA	Digilent BASYS3 (Artix-7)
개발 도구	Vivado 2020.2, Vitis 2020.2
Verification Tool	UVM 시뮬레이션, Vitis Serial Terminal(115200bps), Vivado Hardware Manager
형상 관리	GitHub, Google Docs

3. 아키텍처 설계 (Architecture)

3.1. 시스템 구조

시스템은 MicroBlaze가 AXI4-Lite Interconnect를 통해 모든 주변장치 IP를 memory-mapped 방식으로 제어하는 구조이다. 100MHz system clock과 reset button 입력을 받는 MicroBlaze 아래에 GPIOA(FND segment), GPIOB(FND+ button), GPIOC/GPIOD(LED low/high byte), GPIOE(모드 스위치 + SR04 TRIG/ECHO), custom Timer, custom UART, custom I2C Master(LCD1602), custom SPI Master(RC522), Xilinx axi_uartlite(USB terminal)가 slave로 연결된다.

I2C Master는 LCD1602을 4-bit mode로 구동하고, SPI Master는 RFID-RC522과 register 단위로 통신한다. SR04는 별도 IP 없이 GPIOE의 output bit(TRIG)와 input bit(ECHO)를 이용해 소프트웨어(GPIO) 방식으로 제어한다. Timer IP는 1ms interrupt tick을 생성하고, 이 tick이 시스템 전체의 시간 기준(millis)이 되어 Clock 모듈이 0.1초 단위 표시 갱신을 만들어낸다.

interrupt 경로는 각 IP의 intr 신호가 xlconcat을 거쳐 AXI Interrupt Controller(AXI INTC)로 모이고, INTC가 MicroBlaze로 단일 interrupt를 전달하는 구조이다. Timer(In0), custom UART(In1), I2C (In2), SPI(In3)가 순서대로 연결되어 있으며, 기존 Timer/UART interrupt는 보존하고 I2C/SPI interrupt를 추가하는 방식으로 확장하였다.

3.1.1. AXI4-Lite 주소 맵

Block	Base	High	Software macro	역할
axi_uartlite_0	0x40600000	0x4060FFFF	XPAR_AXI_UARTLITE_0_BASEADDR	USB serial terminal, 115200 bps
gpio_0 (GPIOA)	0x44A00000	0x44A0FFFF	XPAR_GPIO_0_S00_AXI_BASEADDR	FND segment 출력
gpio_1 (GPIOB)	0x44A10000	0x44A1FFFF	XPAR_GPIO_1_S00_AXI_BASEADDR	FND digit select 및 board button
gpio_2 (GPIOC)	0x44A20000	0x44A2FFFF	XPAR_GPIO_2_S00_AXI_BASEADDR	LED low byte
gpio_3 (GPIOD)	0x44A30000	0x44A3FFFF	XPAR_GPIO_3_S00_AXI_BASEADDR	LED high byte
timer_0	0x44A40000	0x44A4FFFF	XPAR_TIMER_0_S00_AXI_BASEADDR	1ms system tick
uart_0	0x44A50000	0x44A5FFFF	XPAR_UART_0_S00_AXI_BASEADDR	Custom UART
i2c_master_0	0x44A60000	0x44A6FFFF	XPAR_I2C_MASTER_0_BASEADDR	LCD1602 I2C
spi_0	0x44A70000	0x44A7FFFF	XPAR_SPI_0_BASEADDR	RFID-RC522 SPI
gpio_4 (GPIOE)	0x44A80000	0x44A8FFFF	XPAR_GPIO_4_S00_AXI_BASEADDR	모드 스위치 및 SR04 TRIG/ECHO

3.2. 시스템 기능 (Function)

3.2.1. 동작 모드

SW[2:0] 조합에 따라 모드가 선택되며, 각 모드가 LCD를 출력시킨다.

SW[2:0]	모드	LCD owner	동작 내용
000	Stopwatch	StopWatch	LCD에 Stopwatch 표시, FND는 스톱워치 시간을 출력. BTNU: run/stop, BTND: clear
001	Current Time	LcdTime	Clock의 현재 시간을 LCD에 HH:MM:SS.t(0.1초) 형식으로 표시
010	RFID Attendance	Attendance	RC522에 카드 태그 시 등록 이름과 IN/OUT 시간 표시, 미등록은 Unknown Card + UID 표시
011	SR04 Distance	Distance	LCD에 'SR04 Distance / Press BTNU' 표시, BTNU 입력 시 1회 거리 측정(mm)
100	Time Setting	TimeSetting	USB terminal에서 T=HH:MM:SS 입력으로 현재 시간 설정

3.2.2. Button 명령

모드	BTNU	BTND	BTNL	BTNR
Stopwatch	run / stop	clear	custom UART로 'r' 전송	custom UART로 'c' 전송
SR04 Distance	1회 거리 측정	x	x	x
그 외 모드	x	x	x	x

3.3. 소프트웨어 계층 구조 (Vitis Layer Structure)

Vitis application은 HAL, driver, common, ap의 4개 계층으로 코드를 분리하였다. 디바이스 인터페이스, 핀맵, 공개 설정에 해당하는 상수는 header에 두고, .c 파일은 상태와 함수 동작만 유지하도록 규칙을 정했다.

Application	Stopwatch						
Driver	LED driver	FND driver	Button driver	LCD driver	RFID driver	SR04 driver	Common
HAL	GPIO	GPIO	GPIO	I2C	SPI	GPIO	TMR
HW	LED	FND	Button	LCD	RFID	SR04	

계층	폴더	책임
HAL	src/HAL	AXI register 직접 접근 및 low-level IP 제어
driver	src/driver	HAL 위에 구축된 device 수준 기능
common	src/common	공용 delay 및 interrupt 지원
ap	src/ap	application 모드 로직과 사용자 가시 동작

3.3.1. HAL 계층

모듈	역할
HAL/GPIO	custom GPIO register 접근: 방향(mode), 입력 read, 출력 write
HAL/TMR	custom Timer 제어, 1ms system tick 생성
HAL/UART	custom PMOD UART (JB1/JB2)
HAL/UARTLITE	Xilinx axi_uartlite USB serial terminal
HAL/I2C	LCD1602이 사용하는 custom I2C master
HAL/SPI	RFID-RC522이 사용하는 custom SPI master

3.3.2. driver 계층

모듈	역할
driver/Button	debounce 처리된 Basys3 버튼 상태
driver/FND	4-digit FND multiplex 출력
driver/LED	LED 출력
driver/LCD1602_I2C	LCD command/data write

driver/RFID_RC522	RC522 register 접근, 카드 감지, UID read
driver/ModeSwitch	GPIO에서 SW[2:0] 읽기
driver/SR04	SR04 trigger 펄스 생성 및 echo 시간 측정

3.3.3. application 계층

모듈	역할
ap/StopWatch	기존 stopwatch 상태 머신과 FND 시간 출력
ap/Clock	공유 현재 시간, 0.1초 해상도
ap/LcdTime	LCD 현재 시간 화면
ap/Attendance	RFID 근태 화면과 UID-이름 매핑
ap/Distance	SR04 거리 화면
ap/TimeSetting	T=HH:MM:SS terminal parser
ap/ModeManager	스위치 모드 선택, LCD ownership, terminal 문자 분배

3.3.4. Startup Flow 와 LCD Ownership

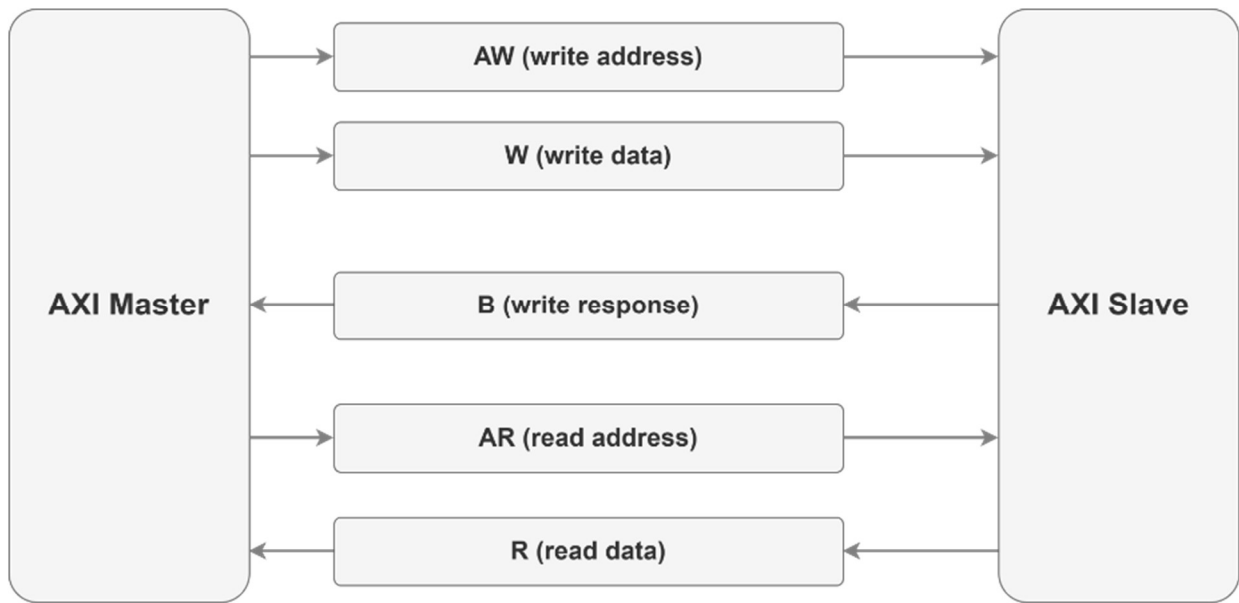
main()의 초기화 순서는 다음과 같다.

(1) StopWatch/Clock 초기화, (2) Timer PSC=99/ARR=999 설정, (3) I2C_Init, (4) SetupInterruptSystem()으로 interrupt controller 기동, (5) custom UART/Timer interrupt enable 및 TMR_StartTimer()로 1ms tick 시작, (6) 그 후에야 LcdTime_Init(), Attendance_Init(), Distance_Init(), TimeSetting_Init(), ModeManager_Init()을 수행하고, (7) while(1) 루프에서 ModeManager_Excute()를 호출한다.

여러 모드가 하나의 LCD를 공유하기 때문에, 현재 선택된 모드만 LCD를 그리도록 ModeManager가 관리한다. ModeManager는 매 loop마다 ModeSwitch_Read()로 스위치 값을 읽고, 값이 변한 경우에만 모드 entry 함수를 호출한다. 새 모드 진입 전에 LcdTime_Disable(), Attendance_Disable(), Distance_Disable(), TimeSetting_Disable()을 모두 호출해 이전 모드의 LCD 소유를 해제한 뒤, 선택된 모드가 화면을 clear하고 다시 그린다. 이를 통해 현재 시간의 주기적 갱신이 RFID/SR04 결과 화면을 덮어쓰는 문제를 구조적으로 방지하였다. 또한 ModeManager는 USB terminal 문자를 모드에 따라 TimeSetting_OnChar(), Attendance_Command() 등으로 분배한다.

3.4. 설계 이론 및 배경 (Theory & Background)

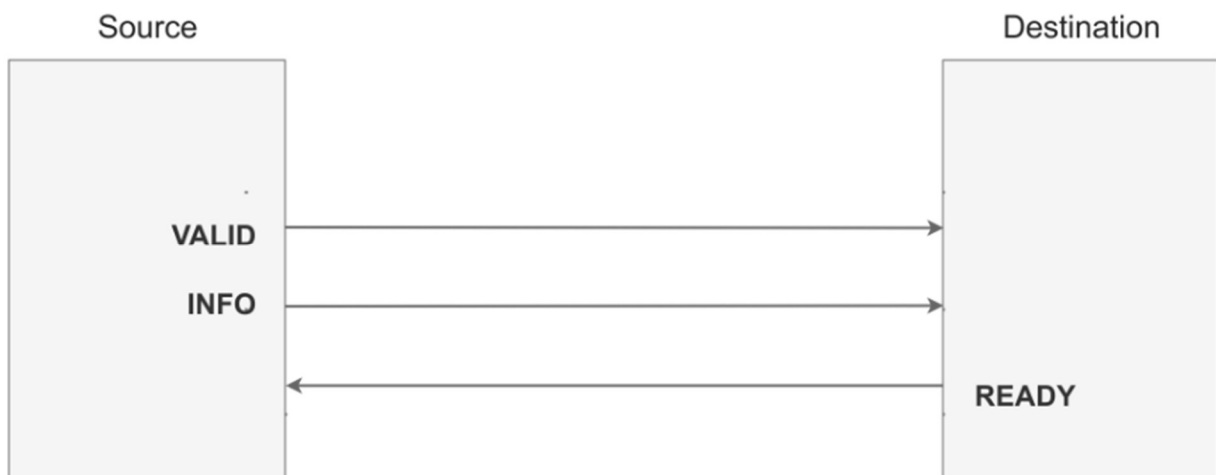
3.4.1. AXI4-Lite 프로토콜



(1) AXI4-Lite 개요

AXI4-Lite는 write address(AW), write data(W), write response(B), read address(AR), read data(R)의 5개 채널로 구성된 handshake 기반 버스 프로토콜이다. burst가 없는 단일 전송만 지원하므로 register 접근 용도의 slave IP를 만들기에 적합하다. 본 프로젝트에서는 MicroBlaze 기반 SoC 구조에서 메모리 맵 (Memory-mapped) 방식의 제어 레지스터를 통해 여러 custom 주변장치 IP를 제어하는 핵심 통신망으로 사용된다.

(2) Handshake 동작 원리



AXI4 프로토콜의 데이터 전송은 Source(송신부)와 Destination(수신부) 간의 비동기적 통신을 동기화하는 핸드셰이크 메커니즘을 기반으로 이루어진다.

Source (송신): 전송할 값(Address, Data 등 제어 정보)을 버스에 올린 뒤 VALID 신호를 1로 구동하여 유

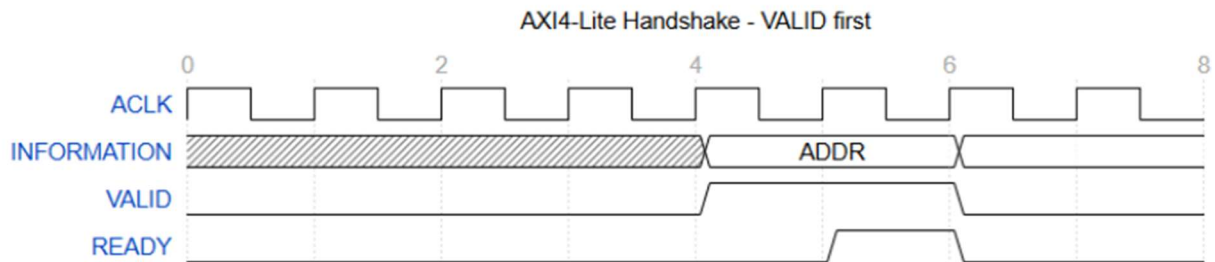
효한 데이터가 준비되었음을 알린다.

Destination (수신): 데이터를 받을 처리 준비가 완료되었을 때 READY 신호를 1로 구동한다.

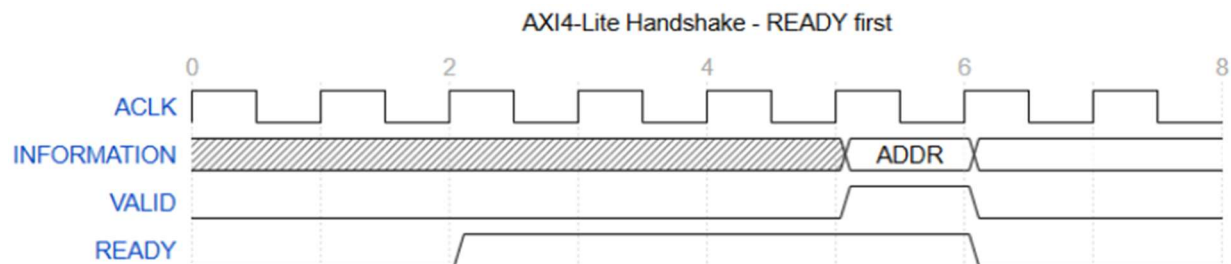
각 채널은 master가 valid를, slave가 ready를 구동하며 두 신호가 동시에 1인 클럭에서 전송이 완료된다.

(3) 전송 타이밍 시나리오

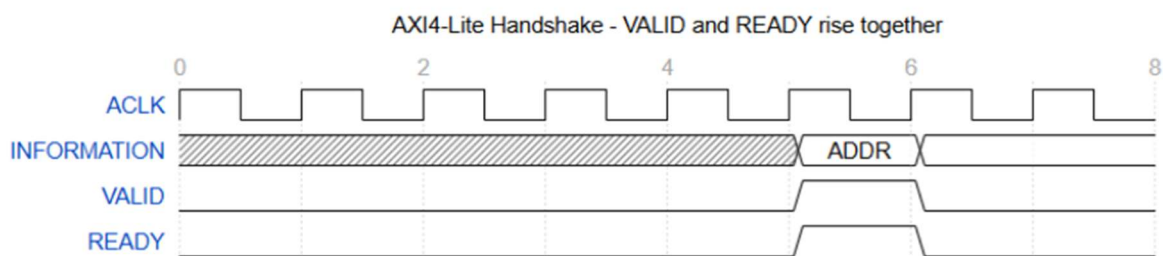
모든 채널은 독립적으로 동작하며, VALID와 READY 신호의 발생 순서에 따라 다음과 같이 세 가지 형태의 유연한 타이밍으로 전송이 완료될 수 있다.



VALID before READY (VALID first): 송신부가 먼저 데이터를 준비하고 VALID를 1로 만들어 대기하고 있으면, 수신부가 준비되었을 때 READY를 1로 올려 전송이 완료되는 방식이다.



READY before VALID (READY first): 수신부가 사전에 READY를 1로 구동하여 수신 대기 상태를 유지하고, 송신부가 데이터를 준비하여 VALID를 1로 올리는 즉시 해당 클럭에서 전송이 완료되는 방식이다.



VALID and READY rise together: 송신부와 수신부가 정확히 동일한 클럭 에지에서 VALID와 READY를 각각 1로 구동하여, 추가 지연 없이 1 클럭 만에 즉시 전송이 완료되는 방식이다.

(4) 트랜잭션(Transaction) 동작 흐름

위의 핸드셰이크 메커니즘을 바탕으로 실제 데이터 읽기/쓰기 동작은 다음과 같은 채널들의 조합으로 이루어진다.

Write 트랜잭션: write는 AW/W 채널 handshake 후 slave가 B 채널로 응답(bresp)을 돌려준다.

Read 트랜잭션: read는 AR 채널 handshake 후 R 채널로 데이터(rdata)와 응답(rresp)이 돌아온다.

3.4.2. I2C 프로토콜

I2C는 SCL/SDA 두 신호를 open-drain으로 공유하는 동기식 직렬 버스이다. master가 SCL high 상태에서 SDA를 falling시켜 START condition을 만들고, 7-bit slave address + R/W bit, 데이터 byte를 전송하며 각 byte마다 slave의 ACK(low)를 확인한다. 전송이 끝나면 SCL high 상태에서 SDA를 rising시켜 STOP condition을 만든다.

3.4.3. SPI 프로토콜과 RC522

SPI는 SCLK/MOSI/MISO/SS 4개 신호를 사용하는 전이중 동기식 직렬 버스이다. master가 SS를 low로 내린 구간 동안 SCLK에 맞춰 MOSI로 데이터를 내보내고 동시에 MISO로 데이터를 받는다. RC522은 register 단위 접근 프로토콜을 정의하는데, address byte와 data byte가 같은 SS low 구간 안에 있어야 하므로, 본 설계의 SPI IP는 SS_HOLD 제어 bit를 두어 다중 byte frame 동안 ss_n을 low로 유지할 수 있게 하였다.

3.4.4. SR04 초음파 거리 측정

SR04는 TRIG 핀에 약 10us의 HIGH 펄스를 주면 40kHz 초음파 8펄스를 송신하고, 반사파가 돌아올 때까지의 시간만큼 ECHO 핀을 HIGH로 유지한다. 음속(약 340m/s) 기준 왕복 시간을 거리로 환산하면 약 58us당 1cm가 되므로, ECHO HIGH 시간을 us 단위로 측정한 뒤 $(echo_us \times 10) / 58$ 식으로 mm 단위 거리를 계산할 수 있다.

3.4.5. UVM 검증 방법론

UVM은 SystemVerilog 기반의 표준 검증 방법론으로, 자극 생성(sequence), 신호 구동(driver), 관찰(monitor), 판정(scoreboard), 커버리지 수집(coverage)을 독립 컴포넌트로 분리한다. sequence는 transaction(seq_item) 단위로 시나리오를 기술하고, driver가 이를 실제 핀 동작으로 변환하며, monitor가 관찰한 transaction을 analysis port로 scoreboard와 coverage에 전달한다. 이 구조 덕분에 자극을 바꿔도 판정 로직을 재사용할 수 있고, directed test와 random test를 같은 환경에서 실행할 수 있다. 본 프로젝트에서는 SoC에서 5회 인스턴스되어 재사용되는 custom GPIO IP를 DUT로 하여 이 구조를 구축하였다.

4. 상세 설계 (Detailed Design)

4.1. Custom GPIO IP

제어 레지스터 (GPIOx_CR)

Address offset: 0x00

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved								CR7	CR6	CR5	CR4	CR3	CR2	CR1	CR0
								rw	rw	rw	rw	rw	rw	rw	rw

Bits [31:8] Reserved, reset 값(0)으로 유지한다.

Bits [7:0] CR: Port 방향 제어

1: 해당 bit output — ODR 값을 io_port로 구동.

0: 해당 bit input — io_port를 high-Z로 두고 외부 값을 IDR로 읽는다.

입력 데이터 레지스터 (GPIOx_IDR)

Address offset: 0x04

Reset value: 0x0000 00XX

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved								IDR7	IDR6	IDR5	IDR4	IDR3	IDR2	IDR1	IDR0
								r	r	r	r	r	r	r	r

Bits [31:8] Reserved

Bits [7:0] IDR: Port 입력 데이터

출력 데이터 레지스터 (GPIOx_ODR)

Address offset: 0x08

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved								ODR7	ODR6	ODR5	ODR4	ODR3	ODR2	ODR1	ODR0
								rw	rw	rw	rw	rw	rw	rw	rw

Bits [31:8] Reserved

Bits [7:0] ODR: Port 출력 데이터

custom GPIO IP(gpio_v1_0)는 AXI4-Lite slave로 4-bit 주소 공간에 4개의 32-bit register를 제공하며, 하위 8-bit가 io_port[7:0]에 매핑된다. 이 중 CR(0x00)/IDR(0x04)/ODR(0x08) 3개가 실제 I/O 기능을 담당하고, 0x0C의 REG3(slv_reg3)는 read/write는 가능하나 I/O 기능이 없는 예비 레지스터로 남겨 두었다. CR의 각 bit가 1이면 해당 bit는 output으로 ODR 값을 io_port에 구동하고, 0이면 input으로 io_port를 high-Z로 두어 외부 구동 값을 IDR로 읽는다. 이 tri-state 구조가 GPIOA~GPIOE 5개 인스턴스에서 동일하게 재사용된다. 각 register의 상세는 아래와 같이 reference manual 형식(offset / reset value / bit field / 접근권한)으로 정리하였다.

4.2. Custom Timer IP

제어 레지스터 (TMR_CR)

Address offset: 0x00

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved														INTR_EN	TMR_EN
														rw	rw

Bits [31:2] Reserved

Bits [1] INTR_EN: Interrupt enable. 1이면 counter가 ARR에 도달할 때 intr 1-clock pulse를 출력한다. 0이면 counting은 유지하되 interrupt는 발생하지 않는다.

Bits [0] TMR_EN: Counter enable. 1: counting 시작, 0: counter 정지(CNT 값 보존).

프리스케일러 레지스터 (TMR_PSC)

Address offset: 0x04

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
PSC[31:16]															
rw															

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PSC[15:0]															
rw															

Bits [31:0] PSC: Prescaler 값. 시스템 클럭을 (PSC+1)로 분주한다. 본 프로젝트는 100MHz를 1MHz tick으로 분주한다.

자동 재장전 레지스터 (TMR_ARR)

Address offset: 0x08

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
ARR[31:16]															
rw															

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ARR[15:0]															
rw															

Bits [31:0] ARR: Auto-reload 값. counter가 ARR에 도달하면 0이 되며 이 시점에 interrupt tick이 생성된다. 본 프로젝트는 1ms 주기를 만든다.

카운터 레지스터 (TMR_CNT)

Address offset: 0x0C

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
CNT[31:16]															
rw															

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CNT[15:0]															
rw															

Bits [31:0] CNT: 현재 counter 값. Read 시 실제 hardware counter(o_cnt)가 반환되고, write 시 counter에 load 된다. SR04 driver의 us 시간 측정(SR04_Micros)이 이 read 경로를 사용한다.

application은 PSC=99, ARR=999로 설정하여 100MHz 클럭에서 정확히 1ms 주기의 interrupt tick을 생성한다. TMR_ISR()은 FND_Excute()로 FND multiplexing을 수행하고 incTick()으로 millis 기반 시간을 누적한다. Clock_Excute()는 이 tick으로부터 100ms 간격을 누적하여 0.1초 표시 필드를 갱신하는데, loop가 한 번 늦게 돌아도 표시 시간이 밀리지 않도록 절대 tick 기반으로 설계하였다. 또한 SR04 driver는 delay_us() 누적이 아니라 Timer CNT를 직접 읽는 SR04_Micros()로 us 단위 시간을 측정하여 측정 정밀도를 확보한다.

4.3. Custom UART IP

상태 레지스터 (UART_SR)

Address offset: 0x00

Reset value: 0x0000 0001

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved														RX_FLAG	TX_READY
														r	r

Bits [31:2] Reserved

Bits [1] RX_FLAG: RX not empty. 1이면 RDR에 읽지 않은 수신 byte가 있다. RDR read 시 clear된다.

Bits [0] TX_READY: TX ready. 1이면 TDR write가 가능하다. 전송 중에는 0.

송신 데이터 레지스터 (UART_TDR)

Address offset: 0x04

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved								TDR[7:0]							
								w							

Bits [31:8] Reserved.

Bits [7:0] TDR: 송신 데이터. TX_READY = 1일 때 write하면 직렬 전송이 시작된다.

수신 데이터 레지스터 (UART_RDR)

Address offset: 0x08

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved								RDR[7:0]							

	r
--	---

Bits [31:8] Reserved.

Bits [7:0] RDR: 수신 데이터. rx_valid 시점에 latch되며 RX_FLAG=1 상태에서 read한다.

제어 레지스터 (UART_CR)

Address offset: 0x0C

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved															RX_IE
															rw

Bits [31:1] Reserved

Bits [0] RX_IE: RX interrupt enable. 1이면 byte 수신 완료 시 intr가 발생하여 UART_ISR이 RDR을 읽는다.

custom UART는 PMOD JB1(tx)/JB2(rx)로 나가는 별도 UART이다. UART_ISR()은 RX interrupt 발생 시 RDR에서 1 byte를 읽어 rx_data에 저장한다. Stopwatch 모드에서 BTNL release 시 'r', BTNR release 시 'c'를 이 UART로 전송하여 외부 장치와의 명령 통신 경로를 유지한다.

4.4. Custom I2C Master IP

제어 레지스터 (I2C_CR)

Address offset: 0x00

Reset value: 0x0000 0002

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved								SOFT_RESET	Reserved		ACK_IRQ_EN	IRQ_ENABLE	RW	STOP	START
								w			rw	rw	rw	rw	w

Bits [31:8] Reserved.

Bits [7] SOFT_RESET: Write 1 시 soft_reset_pulse가 발생해 core를 리셋한다.

Bits [4] ACK_IRQ_EN: ack error interrupt enable.

Bits [3] IRQ_ENABLE: done interrupt enable.

Bits [2] RW: 현재 byte-write 전용이므로 0으로 사용한다.

Bits [1] STOP: Stop enable. 1이면 데이터 ACK 후 STOP condition을 생성한다. Reset 기본 1

Bits [0] START: Write 1 시 start_pulse가 발생하여 전송을 시작하고 done/ack/bus latched flag를 clear한다.

상태 레지스터 (I2C_SR)

Address offset: 0x04

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved											IRQP	BERR	AERR	DONE	BUSY
											r	r	r	r	r

Bits [31:5] Reserved

Bits [4] IRQ_PENDING: (IRQ_STATUS & irq_enable_effective) != 0. 인터럽트 처리 요청 알림

Bits [3] BUS_ERROR: I2C 버스 응답 지연 및 멈춤 에러

Bits [2] ACK_ERROR: 통신 대상 무응답(NACK) 에러

Bits [1] DONE: 전송 완료 latch. START write 시 clear.

Bits [0] BUSY: 전송 진행 중.

송신 데이터 레지스터 (I2C_TXDATA)

Address offset: 0x08

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved								TXDATA[7:0]							
								rw							

Bits [31:8] Reserved.

Bits [7:0] TXDATA: I2C_Slave로 전송할 데이터 byte. START 시점에 core shift register로 load된다.

수신 데이터 레지스터 (I2C_RXDATA)

Address offset: 0x0C

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved								RXDATA[7:0]							
								r							

Bits [31:8] Reserved.

Bits [7:0] RXDATA: I2C read 확장용 레지스터

클럭 분주 레지스터 (I2C_CLK_DIV)

Address offset: 0x10

Reset value: 0x0000 00FA

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CLK_DIV[15:0]															
rw															

Bits [15:0] CLK_DIV: 속도 생성을 위한 clock divider. 하나의 SCL 주기가 4 x CLK_DIV 시스템 클럭이 된다.

슬레이브 주소 레지스터 (I2C_SLAVE_ADDR)

Address offset: 0x14

Reset value: 0x0000 0027

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved								ADDR[6:0]							
								rw							

Bits [31:7] Reserved.

Bits [6:0] SLAVE_ADDR: 7-bit I2C slave 주소. Core가 {ADDR,1'b0}로 address phase를 구성하므로 버스에 0x4E(=0x27 < 1)가 실린다.

인터럽트 허용 레지스터 (I2C_IRQ_ENABLE)

Address offset: 0x18

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved												BEIE	AEIE	DNIE	

Bits [31:3] Reserved.

Bits [2] WRITE_ERROR: READY=0 상태에서 TDR write 시 latch

Bits [1] DONE: Byte 전송 완료,

Bits [0] READY: 1이면 TDR write 가능

송신 데이터 레지스터 (SPI_TDR)

Address offset: 0x04

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved								TDR[7:0]							
								w							

Bits [31:8] Reserved.

Bits [7:0] TDR: 송신 byte. READY=1일 때 write하면 SCLK 8클럭과 함께 MOSI로 shift-out되며 동시에 MISO가 shift-in된다.

수신 데이터 레지스터 (SPI_RDR)

Address offset: 0x08

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved								RDR[7:0]							
								r							

Bits [31:8] Reserved.

Bits [7:0] RDR: 마지막 전송에서 MISO로 수신된 byte. DONE=1 이후 read한다. RC522 register read frame의 두 번째(dummy) 전송에서 실제 register 값이 이 레지스터로 들어온다.

제어 레지스터 (SPI_CR)

Address offset: 0x0C

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved													SSHOLD	CWER	RXIE
													rw	w	rw

Bits [31:3] Reserved.

Bits [2] SS_HOLD: 1인 동안 rfid_ss_n을 low로 유지한다. RC522의 address+data 2-byte frame이 하나의 SS low 구간에 있어야 하므로 register 접근 전 set, 접근 후 clear한다.

Bits [1] CLEAR_WRITE_ERROR: Write 1 시 WRITE_ERROR latch를 clear하는 pulse. 저장되지 않으며 항상 0으로 읽힌다.

Bits [0] RX_IE: RX interrupt enable. 현재 RFID 경로는 polling을 사용하므로 0으로 운용한다.

custom SPI Master IP 는 RC522(RFID)와의 통신을 담당하며, 핵심 특징은 SS_HOLD(CR bit2) 기능이다. RC522 는 주소와 데이터를 하나의 프레임으로 연속 전송해야 하므로, SS_HOLD 로 전송 중 slave select(rfid_ss_n)를 low 로 유지해 프레임이 끊기지 않도록 하였다. 소프트웨어는 1 바이트 전송 전 과정을 처리하는 HAL 함수(SPI_TransferByte)와 이를 감싼 read/write 함수의 계층 구조로 구성했으며, 현재 RFID 경로는 polling 방식으로 동작하고 RX interrupt 는 사용하지 않는다.

5. UVM 검증 환경 설계 (GPIO IP Verification)

5.1. 검증 대상 및 목표

검증 대상(DUT)은 custom GPIO IP(gpio_v1_0)로, CR(0x00)/IDR(0x04)/ODR(0x08)/REG3(0x0C) register와 8-bit tri-state io_port를 갖는다. 검증 목표는 (1) AXI4-Lite write/read 프로토콜의 정상 동작과 bresp/rresp OKAY 응답, (2) wstrb(byte strobe)에 따른 부분 write 동작, (3) CR 방향 제어에 따른 io_port tri-state 동작(출력 bit는 ODR 반영, 입력 bit는 외부 구동 값 IDR 반영), (4) reset 기본값, 방향 토글, 연속 접근, 읽기 전용 IDR write 무시 등 예외 동작의 검증이다.

5.2. Testbench 구성 요소

파일	구성 요소	설명
gpio_if.sv	interface	AXI4-Lite 5채널 신호와 tri 타입 io_port를 포함. ext_drive_en/ext_drive_data로 외부 입력을 bit 단위로 모델링(en=1이면 data 구동, 0이면 high-Z)하고, io_sample로 io_port 상태를 관찰
gpio_pkg.sv	package	scenario enum(RESET_DEFAULT / INPUT_SAMPLE / OUTPUT_DRIVE / MIXED_DIR / ODR_WHILE_INPUT / DIR_TOGGLE / PARTIAL_WSTRB / BACK_TO_BACK / IDR_WRITE_IGNORED / RANDOM) 및 전체 클래스 include
gpio_seq_item.sv	transaction	scenario, cr/odr/ext_drive_data, wstrb, idle_cycles(0~5 제약), expected_/actual_ (cr-odr-idr-io), tr_id, completed 필드를 갖는 uvm_sequence_item
gpio_sequence.sv	sequence	send_gpio() 단일 task를 제공하는 base sequence와 시나리오별 directed sequence, random sequence, full regression sequence
gpio_driver.sv	driver	AXI4-Lite write/read handshake 구현 및 외부 핀 drive 수행, 완료된 transaction을 analysis port로 발행
gpio_monitor.sv	monitor	AXI 프로토콜 감시(reset 중 valid 인가, bresp/rresp OKAY 여부)와 driver가 발행한 결과 신호를 수집하여 transaction을 analysis port로 전달
gpio_scoreboard.sv	scoreboard	driver가 전달한 expected 값과 actual 값 비교로 자동 pass/fail 판정 및 pass/fail 카운트 집계
gpio_coverage.sv	coverage	scenario / direction(CR) / ODR / IDR / wstrb / completed covergroup 수집
gpio_agent.sv / gpio_env.sv /	agent/env/test	sequencer-driver-monitor를 묶은 agent, scoreboard/coverage를 포함한 env, gpio_base_test를 상속한 gpio_basic_test /

gpio_test.sv		gpio_directed_test / gpio_random_test / gpio_full_regression_test
gpio_tb_top.sv	top	100MHz clock 생성, DUT-interface 연결, config_db 설정 및 run_test() 실행(reset 시퀀스는 driver의 apply_reset()이 수행)

5.3. Driver 설계

driver의 axi_write task는 awaddr/awvalid와 wdata/wstrb/wvalid를 동시에 assert하고 awready && wready가 동시에 1이 되는 클럭까지 대기한 뒤 valid를 내리고, bvalid를 기다려 write response를 수신한 후 bready를 내린다. axi_read task는 araddr/arvalid와 rready를 assert하고 rvalid 시점의 rdata를 캡처한다. 외부 입력은 driver가 CR 방향에 따라 ext_drive_en = ~CR로 설정하고 ext_drive_data를 구동하여, 입력으로 설정된 bit를 외부 장치가 구동하는 상황을 재현한다. 각 transaction 완료 시 completed=1과 함께 actual_io와 expected_cr/odr/idr/io값을 채워 analysis port로 발행하므로, scoreboard는 driver 관점의 실제 결과를 그대로 판정에 사용할 수 있다.

5.4. Scoreboard 설계

scoreboard는 driver가 transaction에 실어 보낸 expected_cr/odr/idr/io와 actual 값을 비교하여 자동 pass/fail 판정을 수행한다. 기대값 계산은 driver의 expected_pin_value() 함수가 담당하며, (CR & ODR) | (~CR & ext_drive_data) 식으로 출력 bit는 ODR을, 입력 bit는 외부 구동 값을 따르는 핀 값을 산출한다. driver가 ext_drive_en = ~CR로 입력 bit를 항상 외부에서 구동하므로 high-Z(floating) 구간이 발생하지 않아, 부정값에 의한 false fail이 구조적으로 배제된다.

조건	기대값	비고
CR[i] = 1 (output)	ODR[i]	io_port[i]와 IDR[i] 모두 비교 대상
CR[i] = 0 (input, ext_drive_en[i] = 1)	ext_drive_data[i]	입력 bit는 driver가 항상 구동하므로 비교 대상
ext_drive_en[i] = 0 (floating)	해당 없음	driver가 ext_drive_en = ~CR로 설정하므로 발생하지 않음

CR/ODR/IDR/io_port 4개 항목이 모두 일치하면 pass_count를, 하나라도 불일치하면 uvm_error와 함께 fail_count를 증가시킨다. check_phase에서 검사된 transaction이 없거나 fail_count가 0이 아니면 에러로 처리하고, report_phase에서 pass/fail 카운트를 요약 출력한다.

5.5. Coverage 설계

coverage 컴포넌트는 uvm_subscriber로 driver의 analysis port를 구독하며, 수신한 transaction의 scenario와 expected 값으로 covergroup을 sample한다. coverpoint는 시나리오와 방향/데이터 패턴, byte strobe 조합을 bin으로 명시하여 '설계한 시나리오가 실제로 모두 실행되고 검증되었는가'를 정량적으로 확인할 수 있도록 하였다.

Coverpoint	Bins	의도
cp_scenario	reset_default / input_sample / output_drive / mixed_dir / odr_input / dir_toggle / partial_wstrb / back_to_back / idr_ignored / random	10개 시나리오가 모두 실행되었는지
cp_direction	all_input(0x00) / all_output(0xFF) / low_out(0x0F) / high_out(0xF0) / alternate1(0xAA) / alternate2(0x55) / mixed	전체 입력·출력 및 mixed 방향 구성이 모두 검증되었는지
cp_odr	zero / ones / 0xAA / 0x55 / low / mid / high	출력 데이터 패턴이 전 구간에 걸쳐 인가되었는지
cp_idr	zero / ones / 0xAA / 0x55 / other	입력 합성값 패턴이 관찰되었는지
cp_wstrb	0001 / 0010 / 0100 / 1000 / 1111 / other	byte lane별 strobe 조합이 모두 인가되었는지
cp_completed	completed(=1) / illegal_bins incomplete(=0)	모든 transaction이 완료 상태로 판정되었는지

5.6. Sequence 시나리오

Scenario	내용
GPIO_SCEN_RESET_DEFAULT	reset 직후 CR/ODR이 0x00이고 외부 구동 값이 IDR로 읽히는 기본 상태 확인
GPIO_SCEN_INPUT_SAMPLE	CR=0x00(전체 입력)에서 0x00/0xFF/0xAA/0x55 등 8종 패턴을 외부 구동한 뒤 IDR read
GPIO_SCEN_OUTPUT_DRIVE	CR=0xFF(전체 출력)에서 8종 패턴을 ODR에 write하고 io_port/IDR로 read-back
GPIO_SCEN_MIXED_DIR	0x0F/0xF0/0xAA/0x55 등 8종 방향 mask에서 출력 bit는 ODR, 입력 bit는 외부 값을 따르는 IDR 합성값 검증
GPIO_SCEN_ODR_WHILE_INPUT	전체 입력 상태에서 ODR write가 io_port에 영향을 주지 않음을 확인
GPIO_SCEN_DIR_TOGGLE	CR/ODR/외부 구동 값을 연속 전환하며 방향 토글 시 동작을 확인

GPIO_SCEN_PARTIAL_WSTRB	wstrb=0b0010/0100/1000 부분 strobe write 후 read-back으로 byte lane 단위 갱신 검증
GPIO_SCEN_BACK_TO_BACK	12회 연속 write/read로 handshake 연속 동작 확인
GPIO_SCEN_IDR_WRITE_IGNORED	읽기 전용 IDR(0x04)에 write를 인가해도 무시됨을 확인
GPIO_SCEN_RANDOM	시나리오-wstrb 분포 제약을 둔 랜덤 조합으로 예상하지 못한 조합 탐색

full regression sequence는 위 시나리오들을 순차 실행하는 directed 구간과 제약 랜덤 구간(150회)으로 구성되며, gpio_full_regression_test의 run_phase에서 objection을 잡고 실행된다.

5.7. Testbench Top

gpio_tb_top은 #5 토글로 100MHz clock을 생성하고 gpio_if를 통해 gpio_v1_0 DUT의 AXI 포트와 io_port를 연결한다. AXI 신호 초기화와 reset 시퀀스(aresetn을 5클럭 assert 후 해제)는 driver의 init_bus()/apply_reset() task가 수행하며, tb top은 uvm_config_db로 virtual interface를 전체 계층에 전달한 후 run_test()를 실행한다. 실행할 테스트는 +UVM_TESTNAME 인자로 선택하며, fsdb waveform dump를 함께 수행하여 파형 기반 디버깅이 가능하다.

6. 시뮬레이션 및 검증 (Simulation & Verification)

6.1. GPIO IP UVM 검증 결과

1) 레지스터 기본 접근 및 reset 기본값 확인 (RESET_DEFAULT)

reset 직후 CR/ODR이 0x00으로 초기화되고, 이어지는 write 후 read-back에서 기대값과 실제 rdata가 일치함을 확인하였다. write response(bresp)와 read response(rresp) 모두 OKAY로 수신되었고, monitor의 프로토콜 감시에서도 reset 구간 중 valid 인가나 비정상 응답이 검출되지 않았다.

2) 전체 입력/전체 출력 방향 확인 (INPUT_SAMPLE / OUTPUT_DRIVE)

CR=0x00(전체 입력) 상태에서 tb가 io_port를 구동(ext_drive_en=0xFF)하면 IDR read 결과가 0x00/0xFF/0xAA/0x55 등 외부 구동 값과 일치하였다. CR=0xFF(전체 출력)로 전환 후 동일한 8종 패턴을 ODR에 write하면 io_port와 IDR read 값이 ODR을 그대로 따라가는 것을 확인하였다.

3) Mixed 방향 및 IDR 합성 확인 (MIXED_DIR)

0x0F/0xF0/0xAA/0x55 등 8종 방향 mask 조건에서 출력 bit는 ODR을, 입력 bit는 외부 값을 따르는 IDR 합성 기대값과 실측이 모두 일치하여 pass 하였다. 입력 bit는 driver가 ext_drive_en = ~CR로 항상 구동하므로 high-Z에 의한 부정값 없이 판정이 이루어졌다.

4) Byte strobe 부분 write 확인 (PARTIAL_WSTRB)

wstrb=0b0010/0100/1000으로 CR/ODR의 상위 byte lane만 write한 뒤 read-back하여, strobe되지 않은 하위 byte lane은 이전 값을 유지하고 strobe된 lane만 갱신됨을 확인하였다. 즉 하위 8-bit의 I/O 동작이 상위 lane write에 영향받지 않음을 검증하였다.

5) 방향 토글 및 예외 동작 확인 (DIR_TOGGLE / ODR_WHILE_INPUT / IDR_WRITE_IGNORED / BACK_TO_BACK)

DIR_TOGGLE 시나리오에서 CR/ODR/외부 값을 연속 전환해도 각 단계의 기대값과 실측이 일치함을 확인하였다. ODR_WHILE_INPUT에서는 전체 입력 상태의 ODR write가 io_port에 영향을 주지 않음을, IDR_WRITE_IGNORED에서는 읽기 전용 IDR(0x04)에 write를 인가해도 무시되고 외부 구동 값이 그대로 읽힘을 확인하였다. BACK_TO_BACK에서는 12회 연속 write/read에서도 handshake가 정상 완료되었다.

6) Random 및 Coverage 결과

제약 랜덤 시퀀스를 포함한 full regression 실행 결과 scoreboard fail count 0으로 종료되었고, coverage 컴포넌트의 scenario/direction/ODR/IDR/wstrb/completed coverpoint에서 명시된 bin이 모두 hit 되었다. 특히 cp_completed의 illegal_bins(미완료 transaction)이 한 건도 발생하지 않아, 인가한 모든 자극이 정상적으로 완료·판정되었음을 정량적으로 보일 수 있었다.

6.2. 보드 통합 검증 (Bring-up & Verification)

보드 실행 절차는 (1) Basys3 USB 연결 및 전원 ON, (2) design_1_wrapper.bit로 FPGA program, (3) Vitis에서 Timer_Uart.elf 실행, (4) 115200bps serial terminal 오픈, (5) SW[2:0]으로 모드 선택 순서이다. 각 모드별 기대 동작과 확인 결과는 다음과 같다.

모드	기대 동작	확인 결과
000 Stopwatch	LCD에 Stopwatch 표시, FND 0000 시작, BTNU run/stop, BTND clear	정상. BTNL/BTNR로 custom UART r/c 전송도 확인
001 Current Time	LCD에 현재 시간 HH:MM:SS.t(0.1초) 표시	정상. 100ms 주기 갱신 확인
010 RFID	RC522 정상 시 RFID Ready, 등록 카드는 이름 표시	정상. 등록 2매 이름 표시, 미등록 Unknown Card + UID, 1.5초 중복 쿨다운 확인
011 SR04	SR04 Distance / Press BTNU 표시, BTNU 시 mm 측정	정상. 실측 거리와 경향 일치, 무반사 시 No Echo 표시 확인
100 Time Setting	terminal의 T=HH:MM:SS로 시간 설정	정상. 설정 후 Current Time 모드에서 설정 시간부터 진행 확인

1) 모드 전환 및 LCD ownership 확인

SW[2:0]을 순차 변경하며 모드가 전환되고, 모드 전환 시 이전 모드 화면이 Disable된 후 새 모드가 LCD를 clear하고 다시 그리는 것을 확인하였다. 특히 Current Time의 주기적 갱신이 RFID/SR04 결과 화면을 덮지 않음을 확인하였다.

2) RFID 근태관리 및 진단 확인

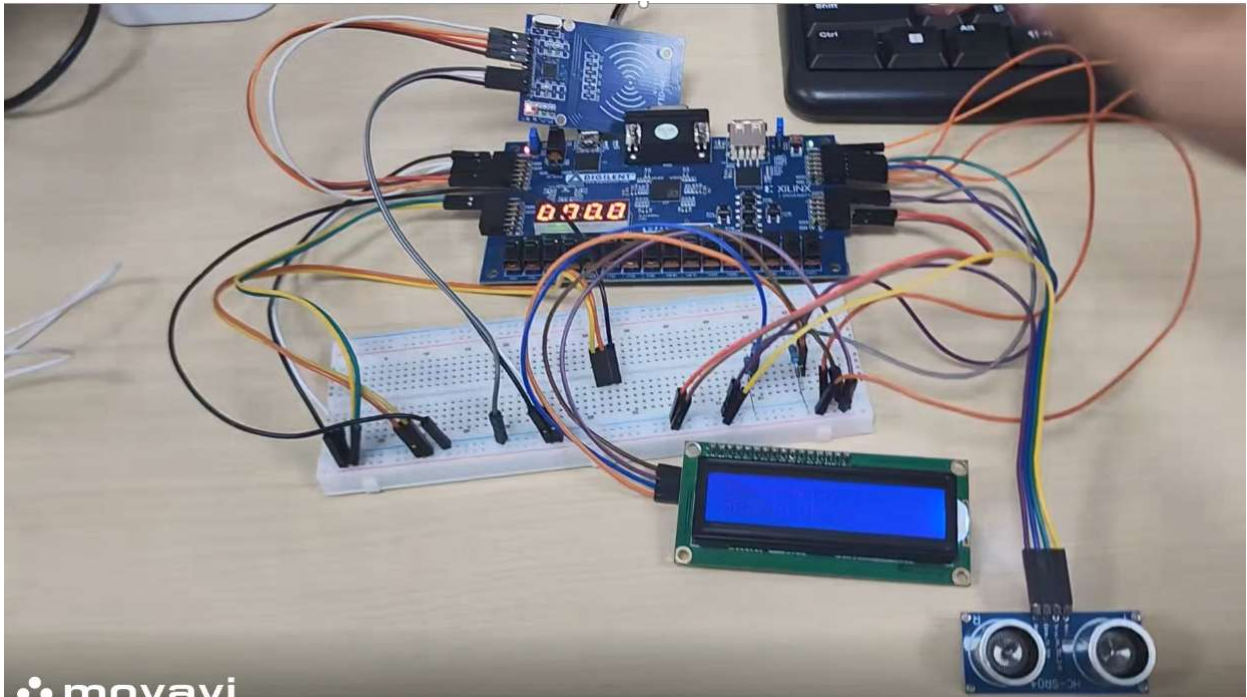
부팅 진단 로그에서 RC522 version이 정상 범위로 읽히는 것을 확인하였다. 등록 카드(2C3C3B06, C7DA2507) 태그 시 이름과 IN/OUT 시간이 표시되는 것을 확인하였다.

3) SR04 측정 확인

BTNU 입력 시 TRIG 10us 펄스 후 ECHO 폭 기반 거리가 LCD에 mm 단위로 표시됨을 확인하였다. 5회 측정 중간값 필터 덕분에 측정값이 안정적이었으며, 장애물 거리를 바꿔가며 측정값 경향이 일치함을 확인하였다.

7. 결과 분석 및 트러블슈팅 (Analysis & Troubleshooting)

7.1. Implementation



Vivado에서 block design 기반으로 synthesis, implementation, bitstream 생성을 완료하고 XSA를 export한 뒤, Vitis에서 platform과 Timer_Uart application을 빌드하여 Launch on Hardware로 실행하였다. 5개 모드 모두 보드에서 정상 동작함을 확인하였다.

7.2. Troubleshooting 1 — AXI 템플릿 Read Data(rdata) 출력 누락

- * 발생 문제: Xilinx AXI4-Lite slave template을 검증하는 과정에서 read transaction을 인가해도 rdata가 전혀 출력되지 않는 상황이 발생하였다.
- * 원인 분석: 템플릿 코드를 추적한 결과, 내부 레지스터 값을 rdata로 전달받는 부분이 코드상에서 지워져 있어 read data path가 끊어져 있었다.
- * 해결 방법: 레지스터에서 rdata를 받아 출력하는 로직을 다시 추가하여 read data path를 복원하였다.
- * 결과: read transaction에서 기대한 레지스터 값이 정상 출력되었고, 이후 custom IP 설계·검증의 참조 기준으로 활용할 수 있었다.

7.3. Troubleshooting 2 — RC522 SPI Address Byte 포맷 규칙 혼선

- * 발생 문제: RC522 통신 규격을 분석하는 과정에서 레지스터 읽기/쓰기 명령에 사용되는 주소 바이트 구성 규칙을 이해하기 어려워, 초기 설계 방향에 혼선이 있었다.
- * 원인 분석: RC522는 단순히 레지스터 주소를 전송하는 방식이 아니라, Write 명령에서는 $(reg < 1) \&$

0x7E 형태로 주소를 구성하고, Read 명령에서는 여기에 최상위 비트(Read 비트)를 설정한 ((reg << 1) & 0x7E) | 0x80 형식으로 전송해야 한다.

* 해결 방법: 데이터시트를 재분석하여 address byte 포맷 규칙을 명확히 정립하고, 이 포맷팅을 사용자 코드가 아닌 드라이버의 레지스터 접근 함수 내부에 고정하였다.

* 결과: 주소 포맷 규칙이 문서와 코드에 일관되게 반영되어, 이후 RC522 레지스터 접근에서 동일한 혼선이 재발하지 않았다.

7.4. Troubleshooting 3 — 모드 전환 시 LCD 화면 글자 겹침

* 발생 문제: 스위치(SW[2:0])를 조작해 다른 모드로 넘어가면, 이전 화면의 글자가 지워지지 않고 새 화면의 글자와 깨진 채로 겹쳐서 출력되는 현상이 발생하였다.

* 원인 분석: 백그라운드에서 0.1초마다 시간을 갱신하던 Current Time 모드가 화면 제어권을 제대로 놓지 않고, 새 모드(예: RFID 화면) 위에 계속 데이터를 덮어쓰고 있었다.

* 해결 방법: ModeManager에서 모드 전환이 감지되는 즉시 이전 모드의 디스플레이 접근 권한을 강제로 박탈(Disable)하고, LCD 화면을 완전히 지운(Clear) 뒤에야 새 모드의 화면을 그리도록 제어권(Ownership) 반환 순서를 엄격하게 수정하였다.

* 결과: 모드 전환 시 화면 겹침 현상이 사라졌고, 이후 통합 과정에서도 모드 간 화면 간섭 없이 안정적으로 동작하였다.

7.5. Troubleshooting 4 — SR04 측정값 불안정 — 타이머 기반 로직으로 전면 수정

* 발생 문제: 초기 설계한 SR04 초음파 거리 측정 코드가 측정값이 자주 튀거나 비정상적인 값을 출력하는 현상이 있었고, 모드 진입 즉시 No Echo 오류가 뜨는 경우도 있었다.

* 원인 분석: 소프트웨어 루프 딜레이를 돌며 ECHO 폭을 재려던 기존 로직이 CPU 오버헤드나 다른 처리의 영향을 받아 정밀도가 떨어졌기 때문이었다.

* 해결 방법: 기존 로직을 폐기하고 하드웨어 Timer의 CNT 레지스터 값을 직접 읽어 마이크로초(us) 단위로 측정하는 타이머 기반 로직(SR04_Micros())으로 전면 수정하였다. 아울러 5회 측정 중간값 필터를 적용하고, 측정 방식도 버튼(BTNU) 트리거 방식으로 개편하였다.

* 결과: 측정값의 신뢰성이 확보되었고, BTNU 입력 시에만 1회 측정하는 구조로 사용성도 정리되었다.

7.6. Troubleshooting 5 — 타이머 고장 — FND 8888 먹통

* 발생 문제: 보드 FND에 8888만 표시되고 타이머 카운트가 전혀 돌아가지 않는 장애가 발생하였다.

* 원인 분석: 전날 코드가 심하게 꼬였을 때 타이머 초기화 및 인터럽트 연동 부분까지 함께 망가진 여파

였다.

* 해결 방법: 꼬였던 타이머 초기화 로직과 실행 순서를 처음부터 다시 정리하여 재구성하였다.

* 결과: 타이머 카운트와 FND 표시가 정상화되었고, 이후 1ms tick 기반 기능들이 안정적으로 동작하였다.

7.7. Troubleshooting 6 — REQA 직후 ProtocolErr 발생

* 발생 문제: 카드 인식 테스트 중 REQA 명령 직후 간헐적으로 프로토콜 에러(ProtocolErr)가 발생하였다.

* 원인 분석: REQA 명령은 마지막 바이트를 7비트만 전송하는 short frame이어야 하는데, 레지스터 설정 (TxLastBits=7)이 누락되어 8비트 풀 프레임이 인가된 것이 원인이었다.

* 해결 방법: TxLastBits 설정을 사용자 API에 맡기지 않고 하위 Request 함수 내부로 강제 편입시켜, 실수 자체가 발생할 수 없도록 구조를 수정하였다.

* 결과: REQA가 정상 short frame으로 전송되어 프로토콜 에러가 사라졌고, 카드 인식이 안정화되었다.

7.8. Troubleshooting 7 — 비트스트림과 XSA(플랫폼) 간 불일치

* 발생 문제: AXI Slave 코드를 다른 프로젝트 코드로 덮어쓰고 비트스트림만 다시 구워 실행했더니 하드웨어가 얼추 동작하는 것처럼 보였으나, UART 시리얼 모니터가 아예 먹통이 되는 현상이 발생하였다. 또한 전날 동작하던 SPI 루프백 통신이 동일 조건에서 재현되지 않는 문제도 함께 겪었다.

* 원인 분석: 비트스트림만 교체하면 된다고 착각한 것이 원인이었다. UART 인터럽트 구성 등 하드웨어 스펙이 바뀌었는데도 XSA를 갱신하지 않아, Vitis 플랫폼(BSP)과 실제 하드웨어의 인터럽트 구조가 어긋나 있었다.

* 해결 방법: 하드웨어 스펙이나 인터럽트 구조가 조금이라도 바뀌면 비트스트림만 굽는 것이 아니라, 하드웨어를 다시 Generate하고 XSA로 Export하여 Vitis 플랫폼을 완전히 갱신하는 절차를 원칙으로 정립하였다. 재현되지 않던 SPI 통신은 코드를 새로 정리하고 재연결하여 검증하였다.

* 결과: 소프트웨어와 하드웨어의 핀트가 맞아 UART/SPI가 정상 동작하였고, 플랫폼 갱신 절차가 문서화되어 동일 실수의 재발을 방지하였다.

7.9. Troubleshooting 8 — 부팅 순서 오류에 따른 시스템 락업

* 발생 문제: 통합 펌웨어를 올리자 전체 시스템이 무반응 상태가 되었다.

* 원인 분석: LCD/RFID 초기화가 인터럽트를 기다리는데 인터럽트 기동이 늦어 멈추는 데드락 현상이었다. 초기화 코드가 interrupt controller와 타이머 시작보다 먼저 실행되고 있었다.

* 해결 방법: SetupInterruptSystem()과 타이머 Start를 main() 최우선 순서로 옮기고, 이후에

LCD/RFID/SR04 application 초기화를 수행하도록 부팅 순서를 재설계하였다.

* 결과: 부팅 즉시 전체 시스템이 정상 기동하였고, 초기화 순서 원칙(인터럽트/타이머 우선)이 문서에 반영되었다.

7.10. Troubleshooting 9 — RFID SPI FAIL (VER=0x00)

* 발생 문제: 부팅 시 LCD에 RFID SPI FAIL, 터미널에 VER=0x00이 출력되며 RC522와의 SPI 통신이 되지 않았다.

* 원인 분석: SoC 내부 버스는 이상이 없었고, RC522 모듈의 SDA 핀을 I2C용 핀으로 착각하여 잘못 배선한 것이 원인이었다. RC522의 SDA는 실제로는 SPI slave select(SS) 역할이다.

* 해결 방법: SDA(SS) 핀을 SPI SS 역할에 맞게 JA4 포트에 교정하고(SCK→JA1, MOSI→JA2, MISO→JA3), VersionReg(0x37) 기반 진단 덤프를 부팅 로그에 연결하여 재발 시 원인을 빠르게 판별할 수 있게 하였다.

* 결과: version이 정상 범위로 읽히며 통신이 복구되었고, 카드 인식이 정상 동작하였다.

7.11. Troubleshooting 10 — SR04 역배선 및 다중 센서 연결 시 LCD 미점등

* 발생 문제: SR04가 동작하지 않았고, 여러 센서를 함께 연결하자 LCD 백라이트가 꺼지는 현상이 발생하였다.

* 원인 분석: SR04의 TRIG/ECHO가 서로 반대로 배선(JB3/JB4 역배선)되어 있었고, LCD 미점등은 다중 센서 동시 연결로 인한 전원/GND 분배 부족과 불완전한 common ground에 따른 전압 강하가 원인이었다.

* 해결 방법: TRIG/ECHO를 정방향(JB3/JB4)으로 교정하고, 접퍼 와이어를 보강하며 보드 메인 접지와 모든 센서의 GND를 하나로 묶는 Common Ground 통합 배선 작업을 수행하였다.

* 결과: 전압 강하가 사라져 LCD가 안정적으로 점등되었고, SR04 거리 측정도 정상 동작하였다.

8. 결론 및 고찰 (Result & Discussion)

8.1. 결론

본 프로젝트는 MicroBlaze와 AXI4-Lite 기반 SoC 통합 설계를 완료하였다. GPIO, Timer, UART, I2C Master, SPI Master의 5종 custom IP를 독자 설계하여 AXI Interconnect에 배치하고, 대표 응용인 SPI RFID 근태관리에 SR04 초음파 거리 측정, Stopwatch 등 응용 기능을 더해 SW[2:0] 기반 모드의 단일 application으로 통합하였다. HAL/driver/common/ap 4계층 구조로 하드웨어 접근과 application 로직을 분리하였고, ModeManager의 Ownership 기반 리소스 관리로 모드 간 LCD 화면 충돌을 구조적으로 방지하였다. 통합 검증에서는 등록/미등록(Unknown) 카드 분기 처리와 1.5초 쿨다운의 중복 태그 방지 로직까지 기획했던 모든 시나리오가 유기적으로 동작함을 확인하였다.

검증 측면에서는 custom GPIO IP에 대해 SystemVerilog UVM 검증 환경을 구축하였다. AXI 5채널 타이밍에 맞춘 Driver와 가상 인터페이스(gpio_if)를 설계하고, Driver가 CR 방향에 따라 기대 핀 값을 산출해 Scoreboard가 CR/ODR/IDR/io_port를 일괄 비교하는 구조로 판정 신뢰성을 확보하였다. 입력 bit를 항상 외부에서 구동하도록 설계하여 high-Z 부정값에 의한 오탐지를 원천 배제하였고, 10종 시나리오를 연속 인가한 Regression 테스트에서 Fail 0건, Coverage 100%를 달성하였다.

기술적으로는 AXI4-Lite 프로토콜의 5개 채널(AW/W/B/AR/R)과 valid/ready 핸드셰이크를 파형 수준에서 이해하고, C 구조체와 RTL 레지스터 맵을 1:1로 매핑하는 HAL 설계, UVM 테스트벤치 구축 경험을 통해 하드웨어-소프트웨어 통합 설계 역량을 확보하였다. 아울러 5개 custom IP와 RC522의 총 34개 레지스터를 집대성한 데이터시트 형식의 레지스터맵 Reference Manual을 작성하여, 개발 산출물이 그대로 재사용 가능한 문서 자산이 되도록 정리하였다.

8.2. 고찰 및 개선 방안

8.2.1. 한계 및 아쉬운 점

첫째, 현재 드라이버는 통신 완료를 확인하기 위해 CPU가 밀리초 단위로 무한 루프를 도는 폴링(Polling) 기반 구조로, 하드웨어 인터럽트와 연동한 비동기 처리까지는 도달하지 못했다. 둘째, Logic Analyzer를 SPI 외부 물리 핀(SCLK, MOSI, MISO, SS)에 직접 연결하여 보드에서 방출되는 실제 프레임 파형을 캡처하는 검증을 고민만 하고 수행하지 못한 점이 아쉬움으로 남는다. 셋째, 카드 사용자 정보가 드라이버 소스 코드의 정적 배열로 고정되어 있어 런타임에 사용자를 관리할 수 없다는 한계가 있다. 이번에 UVM 환경을 GPIO IP에 대해서만 구축한 점도 검증 범위 측면의 한계로 남았다.

8.2.2. 향후 계획 및 개선 방향

향후에는 실전 제품 수준으로 시스템을 고도화하고자 한다. 우선 통신 완료 처리를 하드웨어 인터럽트 신호와 연동하여 백그라운드에서 비동기로 처리하는 인터럽트 기반 드라이버로 개선하고, 레지스터를

한 바이트씩 접근하는 현재 방식 대신 다중 바이트 연속 읽기(FIFO Burst)를 적용하여 긴 UID를 읽을 때의 프레임 낭비를 줄일 계획이다. 또한 UART 시리얼 터미널을 통해 런타임에 사용자를 동적으로 등록·삭제하는 구조로 확장하여 실제 완제품 수준의 근태관리 장비로 발전시키고자 한다. 검증 측면에서는 I2C/SPI 등 나머지 IP로 UVM agent를 확장하고, regression 실행과 coverage 수집을 스크립트로 자동화하는 체계를 갖추면 변경에 대한 안전망이 한층 강해질 것으로 기대된다.